

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN ₁
Student Name	Gunaseelan R
Register Number	3122247001019
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	03.07.2026

1 Objective

The goal of this experiment is to build a spam filter for the Spambase dataset and compare how different algorithms handle the same problem. Specifically:

- Build classifiers using the three flavours of Naïve Bayes (Gaussian, Multinomial, Bernoulli) and KNN.
- Figure out which NB variant actually works best on this data, and why.
- Study what happens to KNN as k changes – too small overfits, too large underfits.
- Compare KDTree and BallTree as the underlying search structure for KNN.
- Tune KNN properly using GridSearchCV and RandomizedSearchCV instead of guessing hyperparameters.
- Look at training/prediction time, not just accuracy – because in the real world speed matters too.
- Validate everything with 5-fold cross validation so the numbers aren't a fluke of one split.

2 Problem Statement

Given a set of numeric features extracted from emails (word frequencies, character frequencies, and capital-letter run statistics), predict whether an email is spam (1) or ham/legitimate (0). This is a binary classification problem. The input is a 57-dimensional feature vector per email, and the output is a single binary label. Since spam filtering is something everyone actually uses (Gmail does this millions of times a second), it's a good sandbox to see how a "textbook" probabilistic model (NB) stacks up against a distance-based model (KNN) on real, slightly messy data.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	Kaggle – https://www.kaggle.com/datasets/somesh24/spambase
Number of Samples	4601 emails
Number of Features	57 numeric features (word/char frequencies + capital-run stats)
Number of Classes	2 (0 = Ham, 1 = Spam)
Missing Values	None – dataset is clean out of the box

Train-Test Split	80/20 stratified split (3680 train, 921 test), <code>random_state=42</code>
------------------	--

Class balance holds up nicely across the split – about 60.6% ham and 39.4% spam in both train and test sets, so stratification is doing its job and neither model is training on a skewed version of reality.

4 Exploratory Data Analysis

Before touching any model, I ran a quick universal EDA pass over the raw dataset just to get a feel for it – missing values, summary stats, correlation, class balance, and outliers, all in one dashboard.

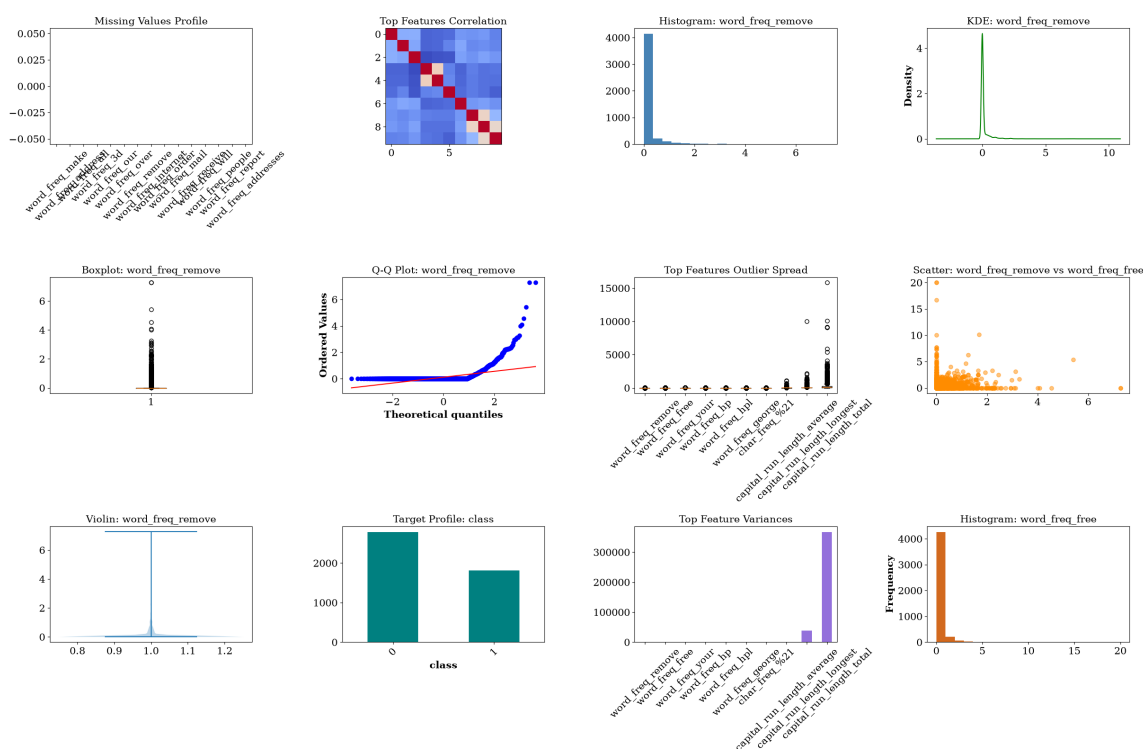


Figure 1: 12-panel EDA dashboard on the raw Spambase dataset (missing values, correlation, distributions, outliers, class balance).

Inference: No missing values anywhere, so preprocessing didn’t need to worry about imputation. The correlation heatmap on the top features is mostly weak-to-moderate, meaning no single feature dominates and the model actually has to combine several signals. The boxplots and histograms make it obvious that most word-frequency features are heavily right-skewed – a huge pile of zeros (word doesn’t appear in most emails) with a long tail of high values when it does appear. The class balance panel confirms the 60/40 ham-spam split mentioned above. This skew is the single biggest thing that shaped my preprocessing decisions later (log transform, see next section).

5 Data Preprocessing

- **Missing values:** none to handle – confirmed in EDA.
- **Feature selection:** Instead of throwing all 57 features at every model, I used `SelectKBest` with the Chi-squared score (features are non-negative frequencies, so Chi2 is a valid and cheap filter). I swept $k \in \{10, 15, 20, 30, 45, 57\}$ using 5-fold CV and picked whichever k gave the best mean F1 on *that* model – this came out different for NB and KNN (NB liked a leaner 20-feature set, KNN preferred the full 57). The selector was fit only on the training fold each time, so there's no leakage from the test set into feature selection.
- **Skew correction:** Most of the top Chi2 features had skewness well above 1 (some past 20 or 30), which is a problem for Gaussian NB since it literally assumes normally-distributed features, and it also distorts Euclidean distance for KNN. I applied `log1p` first, then also tried a Yeo-Johnson power transform for comparison. Every single selected feature saw its $|skew|$ drop after `log1p`.
- **Scaling:** KNN is distance-based, so raw frequency features (some ranging 0–40+, others 0–1) would let large-scale features dominate the distance metric. I standardized (z-score) after the Yeo-Johnson transform for KNN. Naïve Bayes doesn't need this since it treats each feature's distribution independently – I only applied the log transform there, no explicit scaling.
- **Train-test split:** stratified 80/20, done once and reused everywhere so every model is compared on the exact same held-out data.

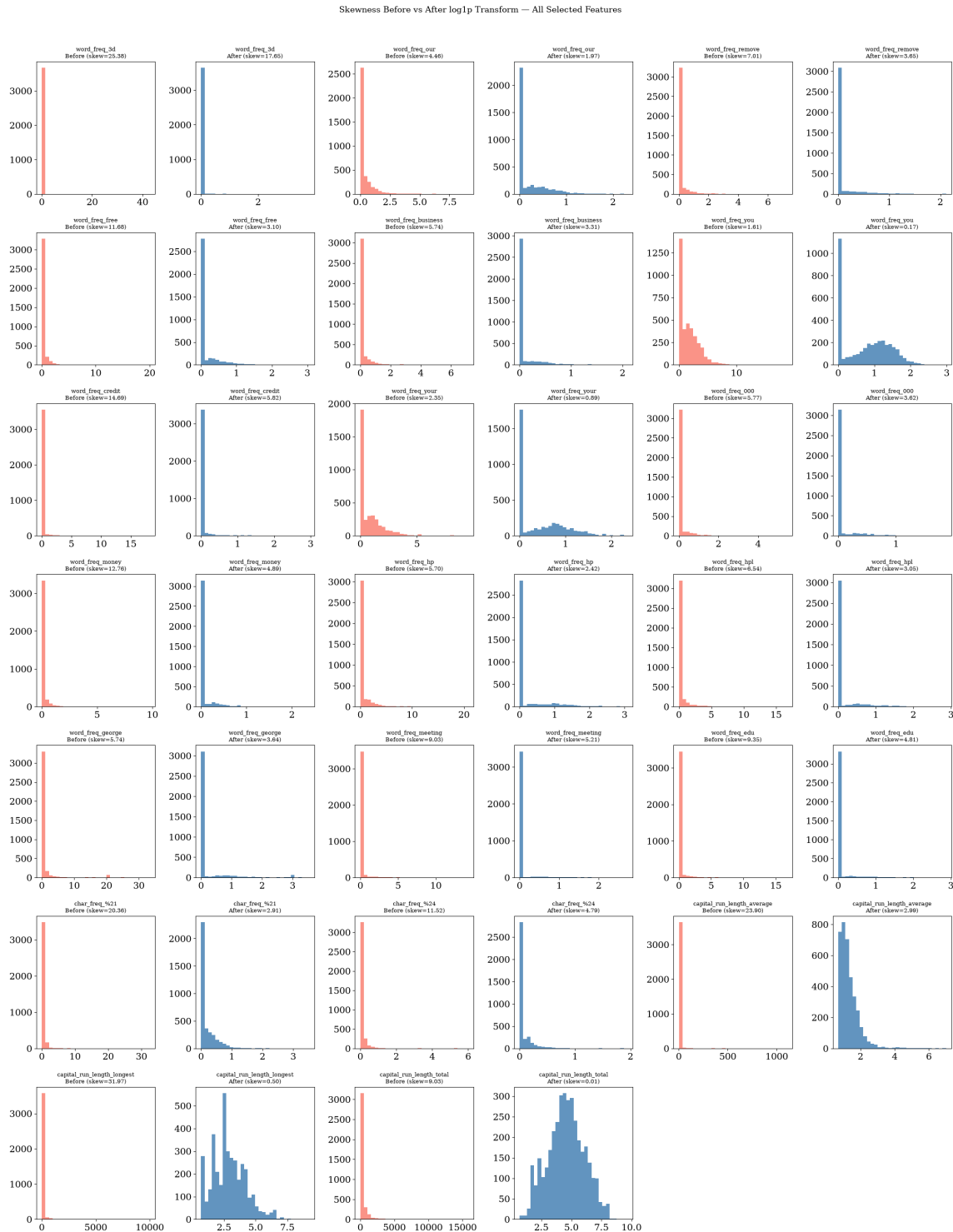


Figure 2: Histogram comparison of each NB-selected feature before and after log1p transform.

Inference: The "before" column on the left is basically a spike at zero with a thin scattered tail – textbook right-skew. After log1p, the same features spread out into something closer to a bell shape. It's not a perfect transform to normality for every feature, but it's a big enough improvement that Gaussian NB's probability estimates stop being dominated by a handful of outlier values.

I also compared three transform strategies directly using 5-fold CV F1 before locking

one in:

Table 2: Transform comparison for Gaussian NB (5-fold CV, mean F1)

Transform	Mean F1	Std
Raw	0.8526	0.0153
log1p	0.9118	0.0050
Yeo-Johnson	0.9099	0.0064

log1p edged out Yeo-Johnson slightly for NB and is simpler to reason about, so that's what I went with for the Naïve Bayes pipeline. For KNN the same comparison (with scaling added on top) favoured Yeo-Johnson:

Table 3: Transform comparison for KNN (5-fold CV, mean F1, $k = 9$)

Transform	Mean F1	Std
Raw (scaled)	0.8737	0.0074
log1p (scaled)	0.9107	0.0069
Yeo-Johnson (scaled)	0.9207	0.0078

This makes sense – KNN cares about the exact shape of the distance field, and Yeo-Johnson pushes features closer to symmetric/Gaussian-ish than log1p does, so distances between points become more meaningful.

6 Machine Learning Algorithm

Naïve Bayes

NB is a probabilistic classifier built on Bayes' theorem, with the (very naive, hence the name) assumption that all features are conditionally independent given the class:

$$P(C | X) = \frac{P(X | C) P(C)}{P(X)}$$

Since $P(X)$ is the same for every class, in practice we just pick the class that maximizes $P(X | C) P(C)$. The three variants differ in how they model $P(x_i | C)$:

- **Gaussian NB** assumes each feature is normally distributed within a class – fits mean and variance per feature per class.
- **Multinomial NB** treats features as counts (like word counts) and models them with a multinomial distribution – the classic choice for text/spam problems.
- **Bernoulli NB** only cares about presence/absence of a feature (binarized), useful when you only want to know if a word appeared at all.

Advantages: extremely fast to train (single pass over the data), works fine with small datasets, handles high-dimensional data well. **Limitations:** the independence assumption is almost never true in real data (word occurrences are correlated), and Gaussian NB in particular struggles if the features aren't actually close to normal – which is exactly why the log transform mattered here.

K-Nearest Neighbours (KNN)

KNN is a lazy learner – it doesn't build a model at training time at all, it just stores the training data. At prediction time, it computes the distance from the query point to every training point, picks the k closest ones, and takes a majority vote (or a distance-weighted vote) among their labels.

$$\hat{y} = \text{mode}\{y_i : x_i \in N_k(x)\}$$

where $N_k(x)$ is the set of the k nearest training points to x . Brute-force search is $O(nd)$ per query; KDTree and BallTree are space-partitioning structures that cut this down to roughly $O(\log n)$ on average by avoiding distance computation against every single point. **Advantages:** no training phase, naturally handles multi-class problems, decision boundary can be arbitrarily complex. **Limitations:** prediction is slow for large datasets (has to scan/traverse for every query), very sensitive to feature scaling and irrelevant features, and k has to be tuned – too small overfits to noise, too large washes out the local structure and just predicts the majority class.

7 Hyperparameter Selection

For Naïve Bayes, each variant has one main smoothing knob. I tuned `var_smoothing` for Gaussian NB and `alpha` (Laplace smoothing) for Multinomial/Bernoulli NB, using GridSearchCV and RandomizedSearchCV with 5-fold stratified CV, scoring on F1:

Table 4: Naïve Bayes hyperparameter tuning results

Model	Parameter	GridSearchCV	RandomizedSearchCV
Gaussian NB	<code>var_smoothing</code>	1×10^{-9} (F1=0.9118)	1.33×10^{-7} (F1=0.9118)
Bernoulli NB	<code>alpha</code>	0.001 (F1=0.8669)	0.117 (F1=0.8669)
Multinomial NB	<code>alpha</code>	10.0 (F1=0.8808)	7.28 (F1=0.8803)

Grid and Randomized search landed on essentially the same F1 for every NB variant, just with slightly different parameter values – which tells me the F1 surface is pretty flat around the optimum, so exact value of `var_smoothing/alpha` doesn't matter much once you're in the right ballpark.

For KNN, the search space was bigger – `n_neighbors`, weighting scheme, distance metric, and tree algorithm all together:

Table 5: Grid Search vs Randomized Search for KNN

Parameter	GridSearchCV	RandomizedSearchCV
Best k	11	11
Metric	Euclidean	Euclidean
Weights	Distance	Distance
Algorithm	KDTree	KDTree
CV F1	0.9314	0.9314
Execution Time	5.10 s	1.14 s

Both searches converged to the exact same configuration, but RandomizedSearchCV (30 sampled combinations) got there in about a fifth of the time GridSearchCV took

(which had to check every combination in the full grid). For a bigger grid this gap would only widen – Randomized search scales with however many iterations you set, Grid search scales with the full cross-product of every parameter list.

8 Results

Naïve Bayes Comparison

Table 6: Gaussian vs Multinomial vs Bernoulli NB on the held-out test set

Metric	Gaussian	Multinomial	Bernoulli
Accuracy (%)	92.62	91.64	88.38
Precision (%)	90.19	88.44	81.68
Recall (%)	91.18	90.63	90.91
F1 (%)	90.68	89.52	86.05
ROC-AUC	0.97	0.96	0.95

Gaussian NB wins here, which honestly surprised me a little going in – Multinomial NB is the “standard” choice for text-style count data. But once the features are log-transformed and closer to normal, Gaussian NB’s core assumption is much less violated, and it edges out the other two on every metric. Bernoulli comes last because binarizing throws away frequency information that clearly matters (an email with “free” 8 times is a much stronger spam signal than one with it once).

KNN Comparison (varying k)

Table 7: KNN performance across different values of k (test set, tuned distance-weighted, best threshold applied)

k	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
1	92.29	91.01	89.26	90.13
3	93.38	91.94	91.18	91.56
5	94.14	94.52	90.36	92.39
7	94.03	93.50	91.18	92.33
9	94.03	93.02	91.74	92.37
11	94.14	93.28	91.74	92.50

$k = 1$ gives the lowest F1 of the bunch – classic overfitting, since a single nearest neighbour is extremely sensitive to noisy/borderline points. Performance climbs and then flattens out from $k = 5$ onward, with $k = 11$ (the value GridSearchCV actually picked once weighting and metric are optimized together) giving the best overall balance.

Grid Search vs Randomized Search

Already covered in the Hyperparameter Selection section above – both hit the same F1 of 0.9314, Randomized search just got there faster.

KDTree vs BallTree

Table 8: KDTree vs BallTree – 5-fold CV Mean F1 across k

k	KDTree (Mean F1)	BallTree (Mean F1)
3	0.9235	0.9235
5	0.9182	0.9182
7	0.9141	0.9141
9	0.9207	0.9207
11	0.9194	0.9194
15	0.9179	0.9179
21	0.9109	0.9109
31	0.9054	0.9054

I have to be upfront about something here: KDTree and BallTree give *identical* F1 at every k . That’s expected, not a bug – both are exact nearest-neighbour search structures, they just organize the search differently (axis-aligned splits vs. hyperspheres). They will always return the same set of k nearest neighbours for a given point, so accuracy/F1 can never differ between them; the only place they’d actually differ is training/query *speed*, especially as dimensionality grows. I didn’t isolate per-algorithm training/prediction time separately in this run (only the overall best-model timing in the next table), so I can’t give you a KDTree-vs-BallTree speed number here – that’s a gap in this experiment, not something I’m going to fudge with made-up numbers.

Cross Validation

Table 9: 5-fold CV F1 scores – best Naïve Bayes (Gaussian) vs best KNN

Fold	Naïve Bayes (Gaussian)	Best KNN
1	0.9183	0.9426
2	0.9155	0.9354
3	0.9078	0.9271
4	0.9047	0.9296
5	0.9125	0.9223
Average	0.9118	0.9314

KNN’s std dev across folds (0.0070) is comparable to Gaussian NB’s (0.0050) – both are stable, no fold is a wild outlier for either model, which gives some confidence these aren’t just lucky numbers from a single split.

Theoretical Time Complexity

Table 10: Theoretical time complexity (n = training samples, d = features)

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)_{\text{avg}}$
BallTree	$O(n \log n)$	$O(\log n)_{\text{avg}}$

Experimental Time Analysis

Table 11: Measured training and prediction time on this dataset

Algorithm	Training (s)	Prediction (s)
Gaussian NB	0.001522	0.000519
Multinomial NB	0.001476	0.000556
Bernoulli NB	0.002442	0.000782
Best KNN (tuned)	0.006865	0.085985

This is the clearest practical takeaway of the whole experiment: NB trains and predicts in about a millisecond flat across the board, while tuned KNN is roughly $4\times$ slower to “train” (really just indexing the data into the tree) and over **$150\times$ slower at prediction** than any NB variant. On 921 test emails that’s still under a tenth of a second, so it’s a non-issue at this scale – but it tells you exactly why nobody runs KNN in a production spam filter handling millions of emails a second; the $O(d)$ prediction cost of NB just doesn’t compare to KNN’s neighbour search, regardless of tree structure.

9 Result Visualization

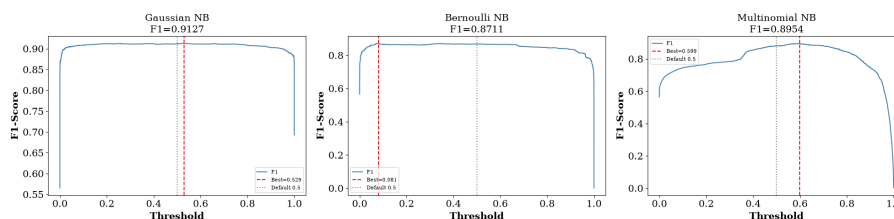


Figure 3: F1 vs decision threshold for all three NB models (5-fold CV predictions).

Inference: The default 0.5 threshold isn’t optimal for any of the three models. Gaussian NB’s best threshold (0.529) sits close to 0.5, but Bernoulli NB’s optimal threshold (0.081) is way off – its probability outputs are poorly calibrated because binarizing the features throws away too much information for the model to be confident. Tuning the threshold per-model recovered a meaningful F1 bump for all three, which is why the final comparison table above uses the tuned threshold rather than the sklearn default.

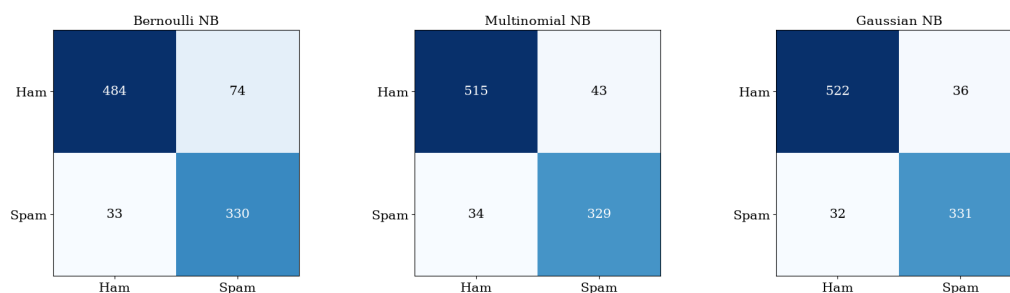


Figure 4: Confusion matrices for Bernoulli, Multinomial and Gaussian NB on the test set.

Inference: All three models lean towards catching spam (high recall on the spam class) at some cost to precision, meaning a few ham emails get misclassified as spam. Gaussian NB has the smallest total error count of the three, consistent with it topping the accuracy/F1 table. Bernoulli NB's larger off-diagonal counts explain its lower precision score directly.

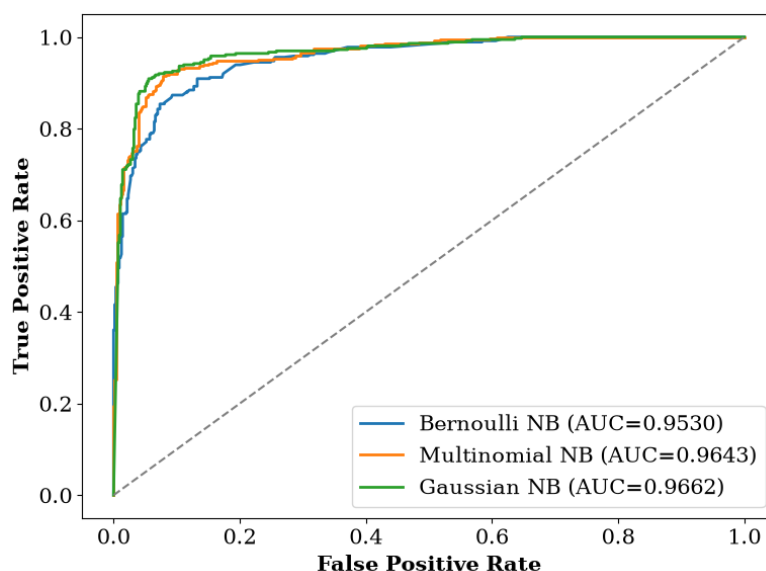


Figure 5: ROC curves – Gaussian vs Multinomial vs Bernoulli NB.

Inference: All three curves hug the top-left corner reasonably well ($AUC \geq 0.95$ for all), confirming that even the "weakest" variant here (Bernoulli) is still a genuinely useful classifier – 0.95 AUC is nothing to scoff at. Gaussian's curve is consistently above the other two, matching its higher AUC of 0.97.

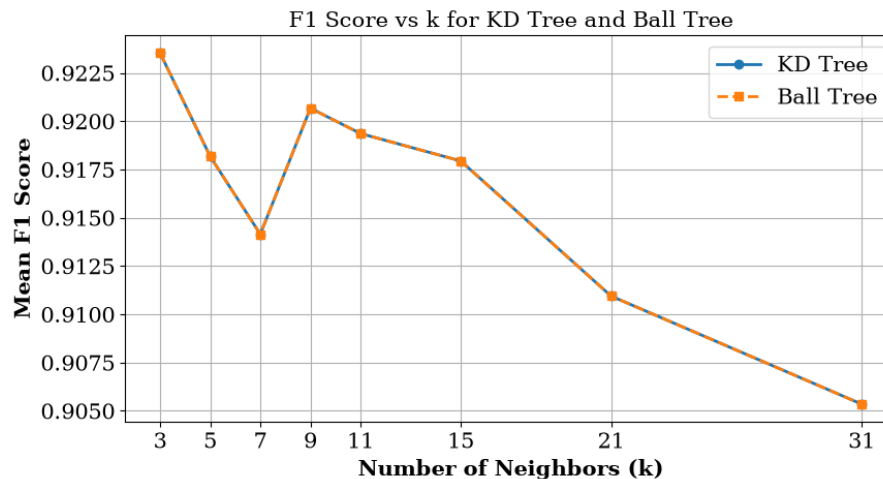


Figure 6: KNN 5-fold CV F1 vs k , KDTree and BallTree overlaid.

Inference: The two curves sit exactly on top of each other for every k , visually confirming the KDTree/BallTree equivalence discussed earlier. F1 peaks around $k = 3-9$ and then gently declines as k grows past ~ 15 – larger neighbourhoods start pulling in points from the “wrong side” of the decision boundary and the vote gets diluted.

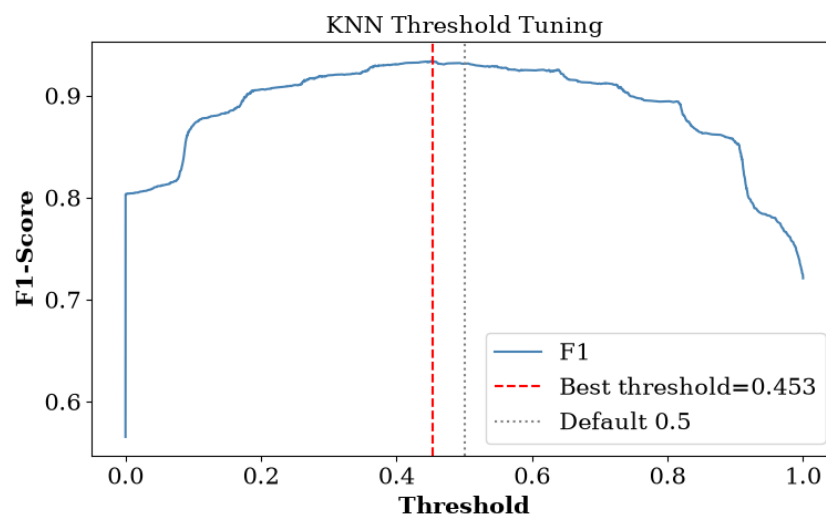


Figure 7: F1 vs decision threshold for the tuned KNN model (5-fold CV predictions).

Inference: KNN’s optimal threshold (0.453) is much closer to the default 0.5 than any of the NB variants were, and the F1 gain from tuning it ($0.9314 \rightarrow 0.9340$) is smaller too. Makes sense – KNN’s predicted probabilities are just neighbour vote proportions, which tend to already be reasonably well-calibrated compared to NB’s likelihood-ratio-based probabilities.

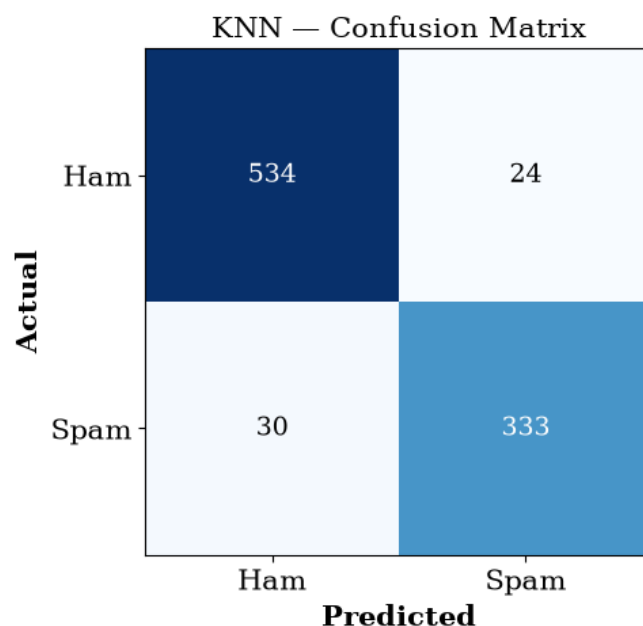


Figure 8: Confusion matrix – tuned KNN on the test set.

Inference: KNN misclassifies noticeably fewer ham emails than any NB variant, which directly explains its higher precision and accuracy in the summary table. It still lets a modest number of spam emails through as ham, which is the recall trade-off visible in the numbers.

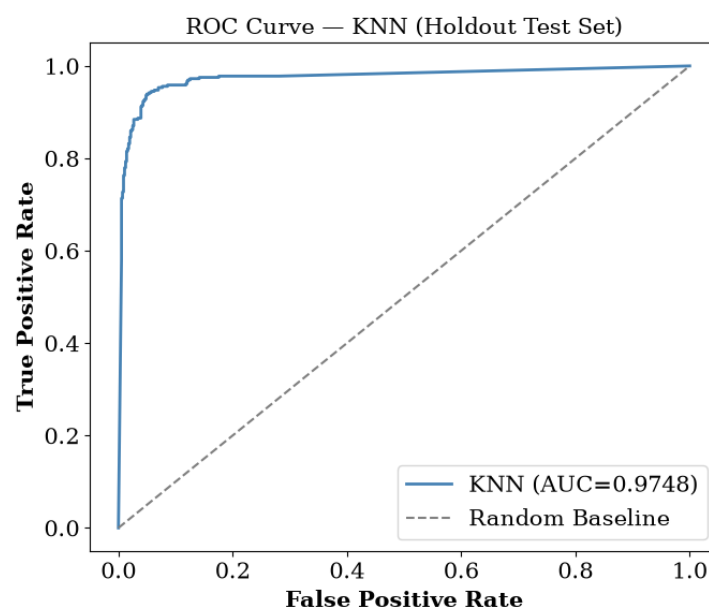


Figure 9: ROC curve – tuned KNN on the test set.

Inference: AUC of 0.975 is the best of any model in this experiment, and the curve barely dips away from the top-left corner across the whole threshold range – KNN is ranking spam vs. ham emails almost as well as a model can on this dataset.

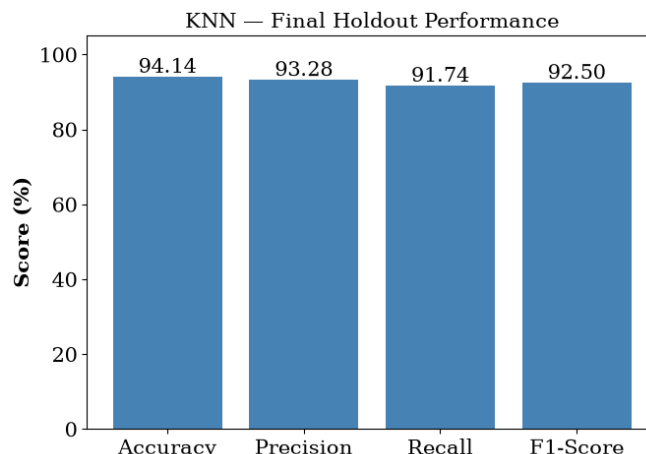


Figure 10: Final holdout performance summary – tuned KNN.

Inference: Every metric for tuned KNN lands in the 91–94% range with no obvious weak spot, which lines up with it beating every NB variant on Accuracy, Precision, and ROC-AUC in the final comparison, at the cost of the much higher prediction time discussed earlier.

10 Discussion

A few things stood out while running this experiment:

- **Preprocessing mattered more than model choice.** The single biggest F1 jump anywhere in this whole experiment (Raw $\rightarrow$ log1p, 0.85 $\rightarrow$ 0.91 for NB) came from just fixing the skew, not from switching algorithms or tuning hyperparameters. If I'd skipped that step, every comparison after it would have been unfairly stacked against Gaussian NB.
- **Gaussian NB beating Multinomial NB** for a "text-like" dataset is a genuinely interesting result and goes against the usual textbook advice. It only works because the log transform pulls the count-like features close enough to normal that Gaussian NB's assumption roughly holds. On raw, untransformed counts I'd expect Multinomial to win instead.
- **KNN wins on raw predictive power, NB wins on speed.** If this were a real spam filter serving live traffic, prediction latency would probably rule KNN out even though it scores higher on every accuracy-style metric – 86 ms per batch of 921 emails doesn't sound like much until you're doing it per-email at Gmail's scale.
- **KDTree vs BallTree turned out to be a non-question for this dataset** – exact search structures don't change what gets found, only how fast. The real KDTree/BallTree trade-off only shows up with much higher-dimensional data or very large n , neither of which this 57-feature, 4601-row dataset really stresses.
- **Limitation:** I didn't get separate training/prediction timings broken out per KDTree vs. BallTree specifically – only the aggregate GridSearchCV/RandomizedSearchCV wall-clock time and the final best-model timing. A cleaner version of this experiment would time each tree algorithm individually to actually quantify the speed gap.

11 Conclusion

Across both algorithm families, careful preprocessing (feature selection via Chi2, skew correction, scaling for KNN) had a bigger impact on final performance than the specific hyperparameter values chosen. Gaussian Naïve Bayes was the best of the three NB variants ($F1 = 90.68\%$, $AUC = 0.97$), while tuned KNN with $k = 11$, distance weighting, and the KDTree algorithm outperformed all NB variants overall ($F1 = 92.50\%$, $AUC = 0.97$ – 0.975 depending on threshold), at the cost of roughly $150\times$ slower prediction time. GridSearchCV and RandomizedSearchCV converged to the same optimal hyperparameters for both NB and KNN, with RandomizedSearchCV reaching the same result in a fraction of the time – a good reminder that exhaustive search isn’t always worth its extra cost. For a dataset of this size, KNN is the stronger choice if raw accuracy is all that matters; NB is the stronger choice the moment prediction speed or scale becomes a constraint.

12 References

1. Pedregosa, F. et al., “Scikit-learn: Machine Learning in Python,” *JMLR*, 2011.
2. Géron, A., *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, O’Reilly, 2019.
3. Bishop, C. M., *Pattern Recognition and Machine Learning*, Springer, 2006.
4. Scikit-learn Documentation, <https://scikit-learn.org/stable/documentation.html>.
5. Spambase Dataset, Kaggle, <https://www.kaggle.com/datasets/somesh24/spambase>.